ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Robotics & Computer Vision

Week 12: Localization & Visual Odometry

Robot self-estimates motion using camera, image features, and encoders — applied to position tracking for Turtlebot 4.

Lê Đình Phong, Ph.D

Week 12 Objectives

1	2	3
Understand Concepts	**Master the Pipeline**	**Python Practice**
Distinguish localization, odometry, dead reckoning, and visual odometry in mobile robots.	Camera → feature detection → matching → motion estimation → pose integration.	Use OpenCV + encoder/camera data to track a robot's 2D trajectory.

Week 12 in the Curriculum

Robot self-estimates distance traveled based on changes in image feature points. Practical application: track robot position on a map using camera/encoder data via Python for Turtlebot 4.

Inputs

Sequential camera frames, wheel encoders, camera parameters.

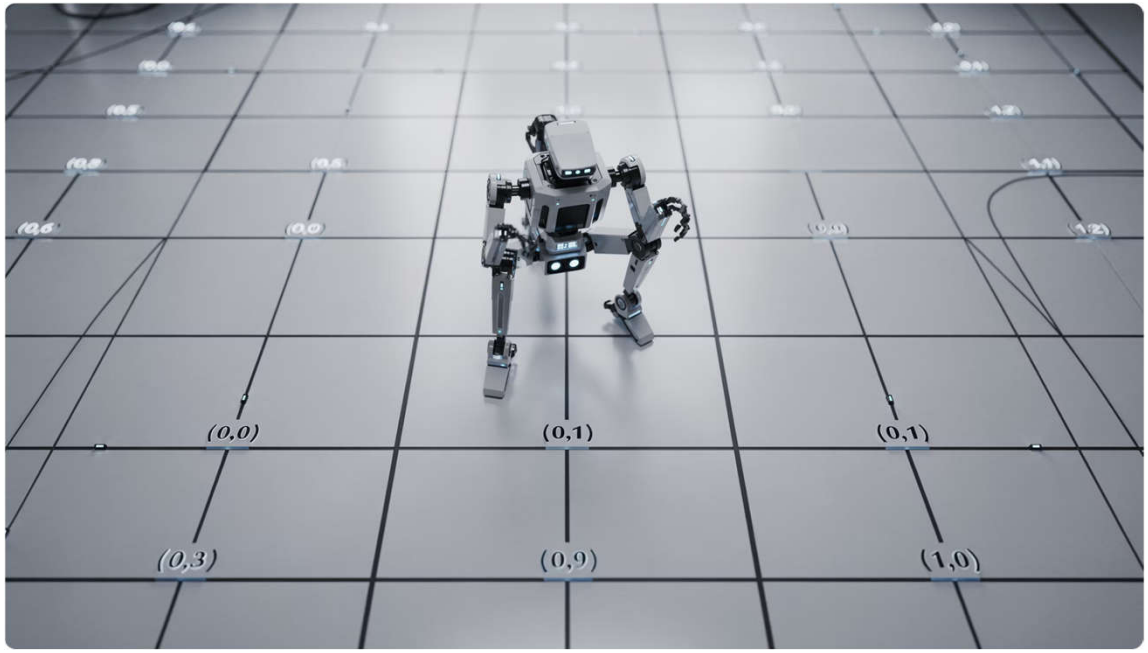
Outputs

2D or 3D pose sequence over time.

Applications

Basic localization, mapping, navigation, SLAM.

What is Localization?



Definition

Localization determines **where a robot is** and **which direction it faces** in a reference frame.

- 2D pose: x, y, theta
- 3D pose: x, y, z, roll, pitch, yaw
- Can be absolute or relative

This week focuses on **relative localization** — accumulating displacement between consecutive frames: Poset = Poset-1 ⊕ ΔPose

What is Odometry?

Wheel Odometry

Estimates motion from wheel encoder readings.

Visual Odometry

Estimates motion from changes in camera images.

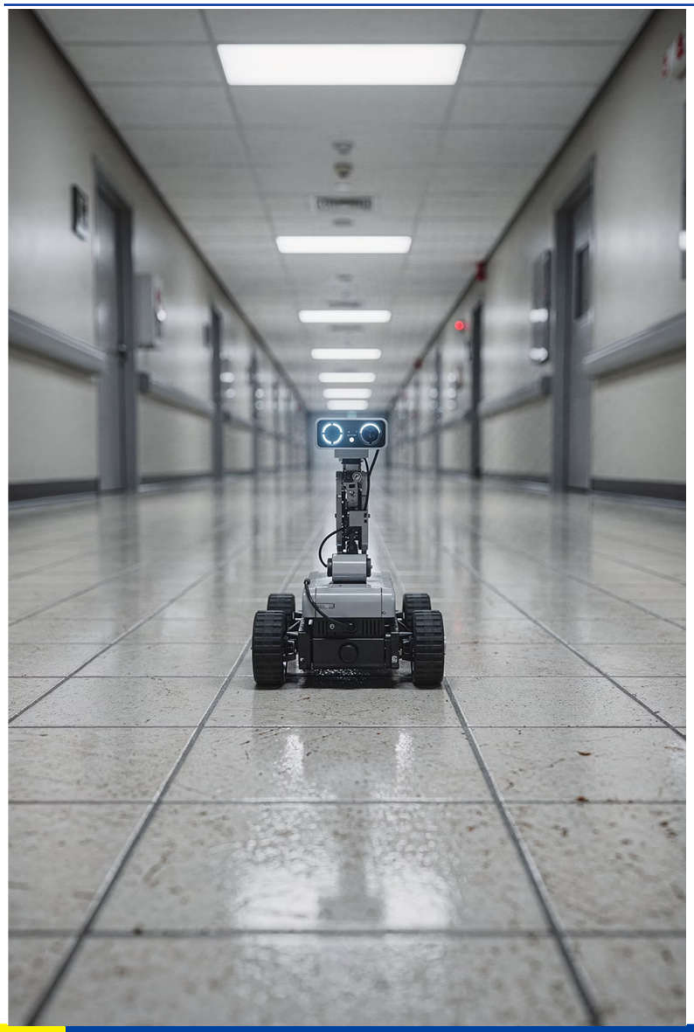
Sensor Fusion

Combines multiple sources to reduce drift and improve stability.

Odometry does not necessarily give absolute position — it tells how the robot has moved **since the last timestep**.

Dead Reckoning vs. Localization

Concept	Meaning	Problem
Dead Reckoning	Accumulates motion from internal sensors	Error grows over time
Localization	Estimates pose in a reference frame	May need landmarks, maps, external sensors
Visual Odometry	Dead reckoning using a camera	Sensitive to texture, lighting, blur



Why Visual Odometry?

- Encoders can slip on smooth or dusty floors.
- IMUs suffer from bias and drift over time.
- GPS is not useful indoors.
- Cameras are cheap, common, and rich in environmental information — a single camera can generate motion cues, especially in textured indoor environments.

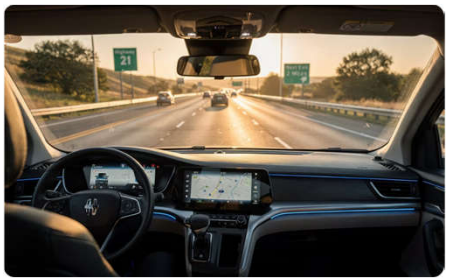
Where is Visual Odometry Used?

Vacuum Robots

Tracks path inside the home.

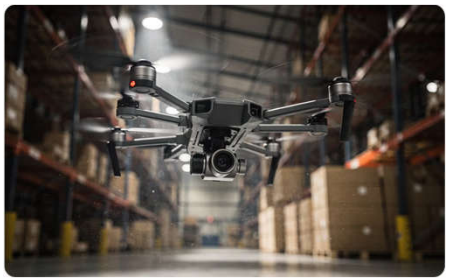
Turtlebot

Maintains pose estimate before full SLAM is available.



Self-Driving Cars

Estimates motion between camera frames.



Drones (VIO)

Visual-Inertial Odometry is widely used when GPS is weak.

The Core Intuition of Visual Odometry

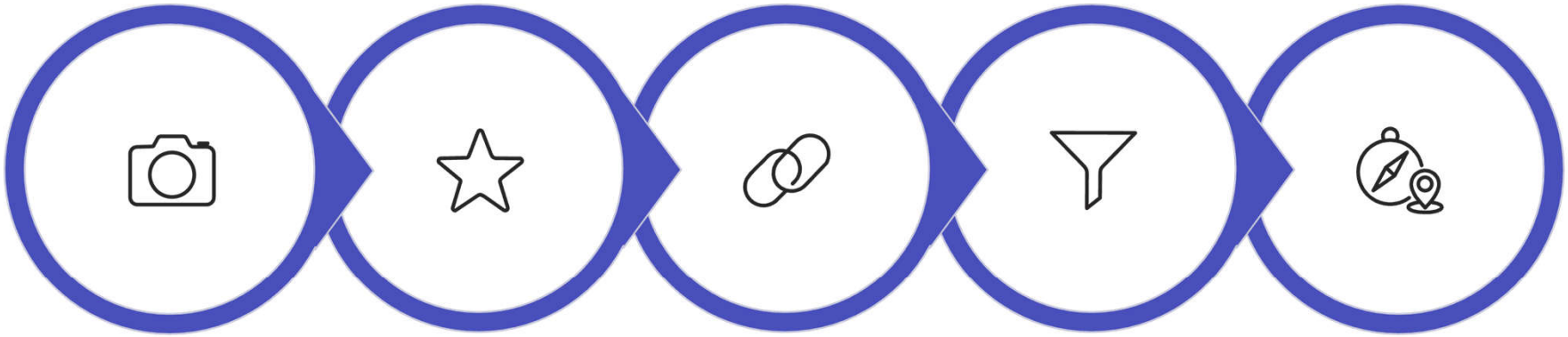
If the Camera Moves...

Feature points in the image also shift. From this shift, we infer how the camera has **translated and rotated**.

Requirements

- Scene must have texture or edges
- Consecutive frames must be close enough
- Blur and rolling shutter must be minimal

The VO Processing Pipeline



Capture Frames | **Detect Features** | **Match / Track** | **Estimate & Filter** | **Recover & Accumulate**

Each step builds on the previous — errors at any stage propagate forward, making robust matching and outlier rejection critical.

Two Approaches: Feature-Based vs. Direct VO

Feature-Based

- Detect corners/keypoints
- Compute descriptors
- Match correspondences
- Easy to teach and debug

Direct

- Uses raw pixel intensities
- Optimizes photometric error
- Strong with smooth textures
- Harder to implement — advanced topic

Why Teach Feature-Based VO First?

Observable

Students can visually see keypoints and match lines between frames.

Codeable

Directly uses OpenCV functions like ORB and BFMatcher.

Math-Connected

Naturally links to epipolar geometry and pose estimation theory.

What Makes a Good Keypoint?

Stable

Does not disappear in the next frame.

Distinctive

Descriptor is unique enough to match correctly.

Well-Distributed

Not clustered in one corner of the image.

Appropriately Sensitive

Not overly affected by lighting changes.

Keypoint Detectors: FAST, Harris, Shi-Tomasi

Harris

Classic corner detector — great for building mathematical intuition.

Shi-Tomasi

Used in `goodFeaturesToTrack` — ideal for optical flow tracking.

FAST

Very fast, suitable for real-time; typically paired with ORB descriptor.

Descriptors: SIFT, SURF, ORB

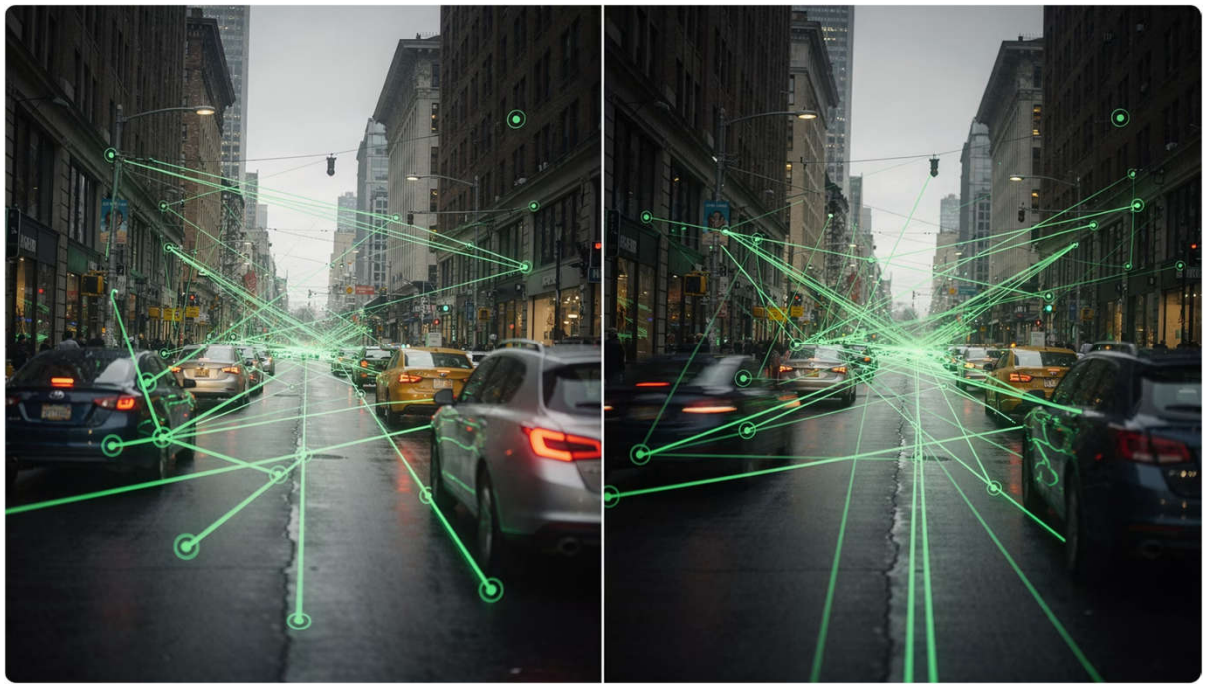
Algorithm	Advantages	Disadvantages
SIFT	Stable, robust to scale/rotation	Slower
SURF	Faster than SIFT in many contexts	Less common in current open builds
ORB	Fast, free, real-time ready	Less robust under strong transformations

Why ORB for Week 12?

- Built into OpenCV — no extra install.
- Fast on standard laptops.
- Binary descriptor enables efficient matching.
- Sufficient quality for basic VO demos.

```
orb = cv2.ORB_create(1000)
kp, des = orb.detectAndCompute(
 gray, None)
```

Brute-Force Feature Matching



Principle

Compare each keypoint descriptor in frame 1 against all descriptors in frame 2 — take the closest pair.

Key Notes

- ORB uses Hamming distance
- Use `crossCheck=True` for mutual best matches
- Sort by distance ascending for best results

Why Remove Outliers?

Wrong Matches

Two points look similar but belong to different objects.

Dynamic Objects

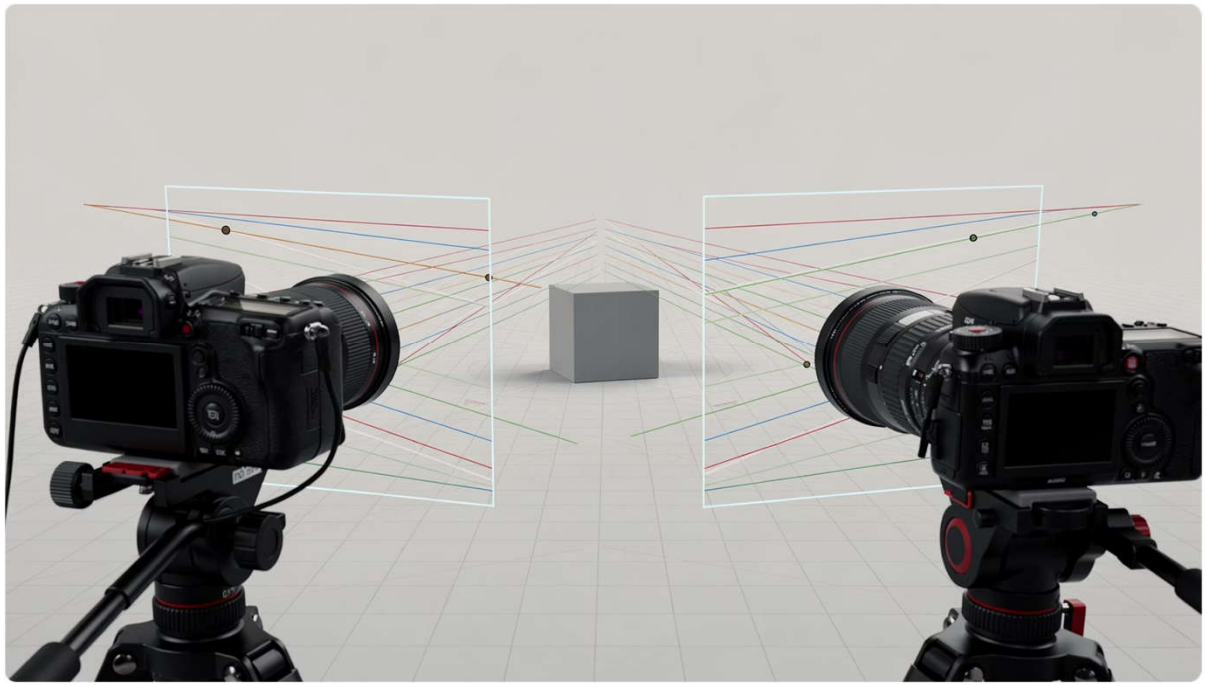
Pedestrians or moving objects create false motion signals.

Blur

Unstable descriptors lead to incorrect pose estimates.

⚠ Without outlier filtering, the geometric matrix will be skewed and accumulated pose will drift rapidly.

Epipolar Geometry — The Intuition



Key Insight

A point in frame 1 cannot appear *anywhere* in frame 2 — it is constrained to an **epipolar line**.

Why It Matters

This constraint lets us verify correct/incorrect matches and recover the relative motion between two camera poses.

Essential Matrix & Fundamental Matrix

F Matrix

Works with pixel coordinates — no need to know the camera intrinsics.

E Matrix

Requires intrinsic matrix **K**; from E we can recover R and t — up to an unknown scale.

$$x'^{T} F x = 0 \quad \text{and} \quad x'^{T} E x = 0$$

Why RANSAC is Critical

Outlier Tolerant

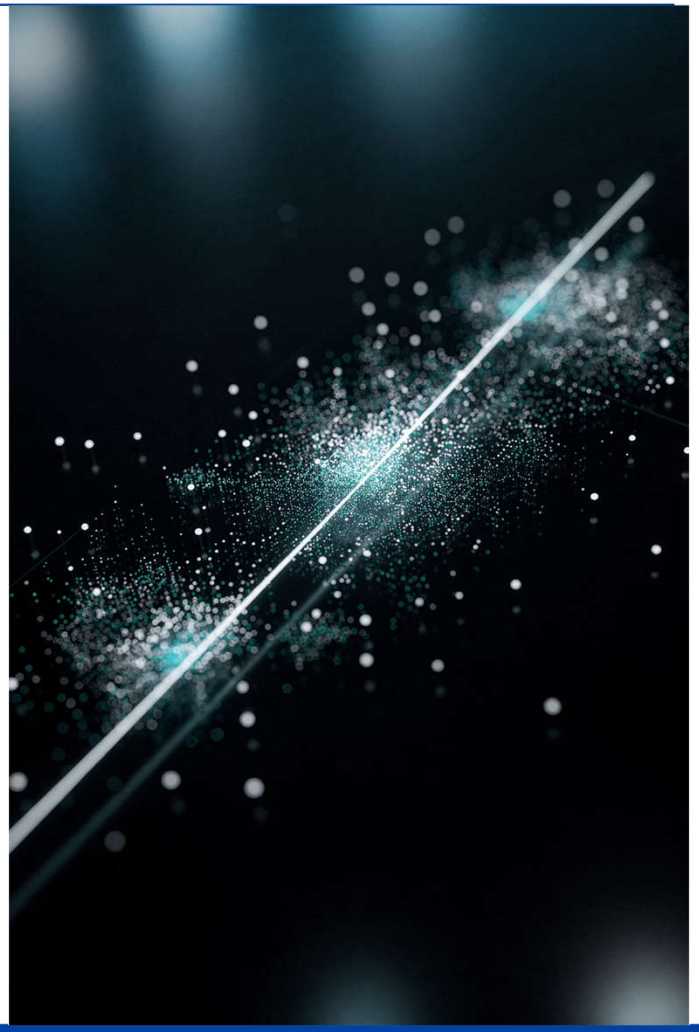
Does not require all matches to be correct.

Robust Estimation

Selects the inlier set that best fits the geometric model.

Practically Essential

Saves the system from bad matches in real-world conditions.



Scale Ambiguity in Monocular VO

⚠ **The Problem:** From a single camera, we recover the direction of translation but not the exact distance traveled in meters — unless we add extra assumptions or sensors.

Solutions

- Use a stereo camera
- Use a depth camera
- Fuse with encoder or IMU
- Use known object sizes as reference

Why Fuse Encoder with Camera?

Encoder Provides Scale

Tells how far the wheels turned — useful to anchor motion magnitude.

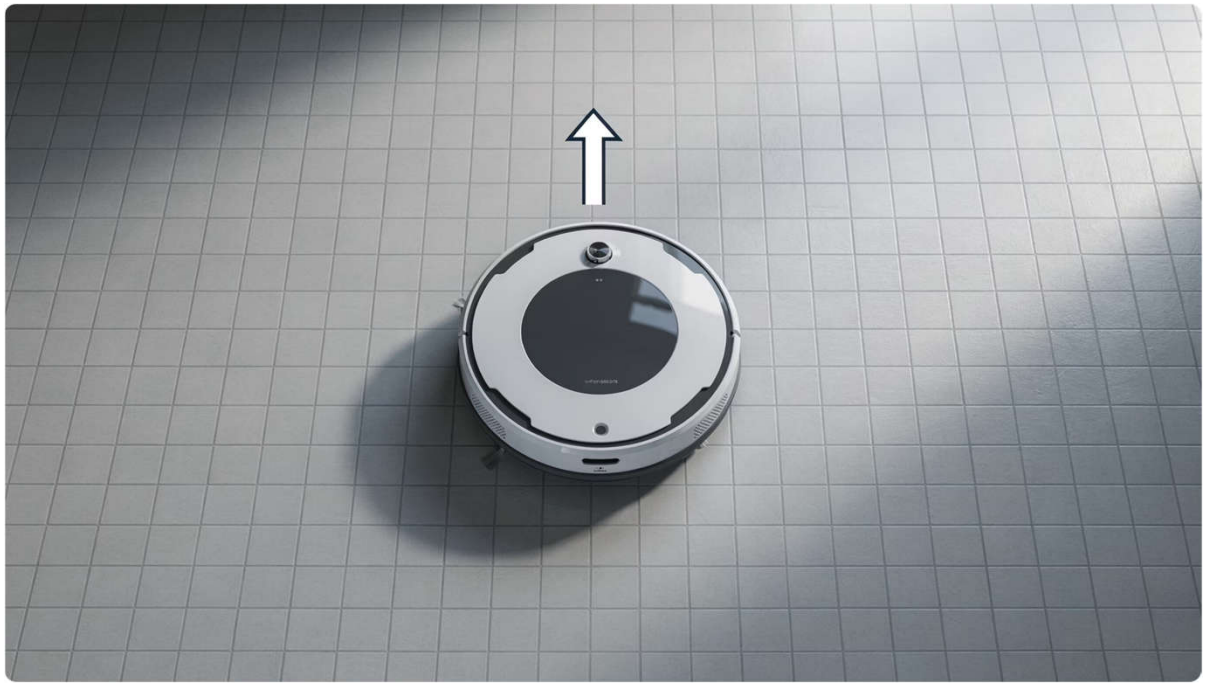
Camera Provides Direction

Supplements when wheels slip or rotate incorrectly.

Combined Result

Smoother pose estimate than either source alone.

2D Pose of Turtlebot



In this lecture we prioritize visualizing the trajectory on the floor plane.

```
pose = (x, y, θ)
```

- **x, y**: position on the map
- **θ**: robot heading direction
- **Δpose**: updated from encoder or VO

Wheel Odometry Update Equations

For a differential-drive robot, the pose update at each timestep is:

$$
\begin{aligned}
\Delta s &= (\Delta s_R + \Delta s_L) / 2 \\
\Delta\theta &= (\Delta s_R - \Delta s_L) / W \\
x &\leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta/2) \\
y &\leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta/2) \\
\theta &\leftarrow \theta + \Delta\theta
\end{aligned}
$$

Where Wheel Odometry Fails

Wheel Slip

Tile floors, dusty surfaces, sharp turns.

Wrong Wheel Radius

Model error accumulates over distance.

Wrong Wheelbase

Causes heading error to grow.

Encoder Noise

Noise and quantization errors compound.

Optical Flow — An Intuitive Entry to VO

Idea

Track good feature points from one frame to the next using **Lucas-Kanade optical flow**.

Teaching Value

Students can directly see motion vectors and connect them to the motion field concept in images.

What Does Optical Flow Reveal?

Forward / Backward Translation

Flow expands outward or contracts inward around the Focus of Expansion (FOE).

Left / Right Rotation

Vectors tilt with a consistent structured pattern.

Independent Moving Objects

Anomalous vectors — must be excluded from the background motion model.

Camera Calibration Matters

- Intrinsic matrix **K** directly affects Essential matrix computation.
- Lens distortion warps straight lines and degrades matching.
- Undistorting images before VO improves stability.

```
K = [[fx, 0, cx],
 [ 0, fy, cy],
 [ 0, 0, 1]]
```

Image Preprocessing Best Practices

1

Grayscale

Faster processing — sufficient for feature detection.

2

Resize

Reduce CPU load for real-time performance.

3

Undistort

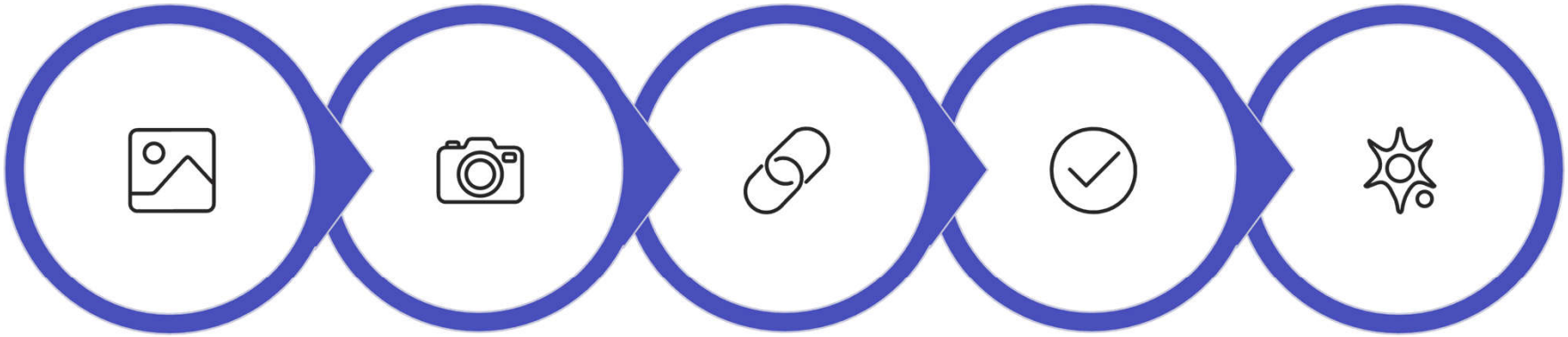
Correct geometric distortion from lens.

4

Mask ROI

Remove ceiling regions or dynamic objects not relevant to motion.

Minimal Monocular VO Pipeline in Python



Read Frames | **ORB Detect** | **BFMatcher** | **Select Matches** | **Essential Mat**

This minimal pipeline is sufficient for a working VO demo — each step maps directly to an OpenCV function call.

Code: Initialize Detector & Matcher

```
orb = cv2.ORB_create(1500)
bf = cv2.BFMatcher(cv2.NORM_HAMMING,
 crossCheck=True)

kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

matches = bf.match(des1, des2)
matches = sorted(matches,
 key=lambda m: m.distance)
```

Code: Extract Point Correspondences

```
pts1 = np.float32(
    [kp1[m.queryIdx].pt for m in matches]
).reshape(-1, 1, 2)

pts2 = np.float32(
    [kp2[m.trainIdx].pt for m in matches]
).reshape(-1, 1, 2)
```

This is the input for geometric matrix estimation. Use only the top-N best matches for improved stability.

Code: Compute Essential Matrix

```
E, mask = cv2.findEssentialMat(
    pts1, pts2, K,
    method=cv2.RANSAC,
    prob=0.999,
    threshold=1.0
)
```

mask identifies which matches are inliers.

Threshold too large → keeps outliers. Too small → loses data.

This is the **most critical step** in monocular VO.

Code: Recover Relative Pose

```
_, R, t, mask_pose = cv2.recoverPose(
    E, pts1, pts2, K
)
```

R

3×3 rotation matrix between the two frames.

t

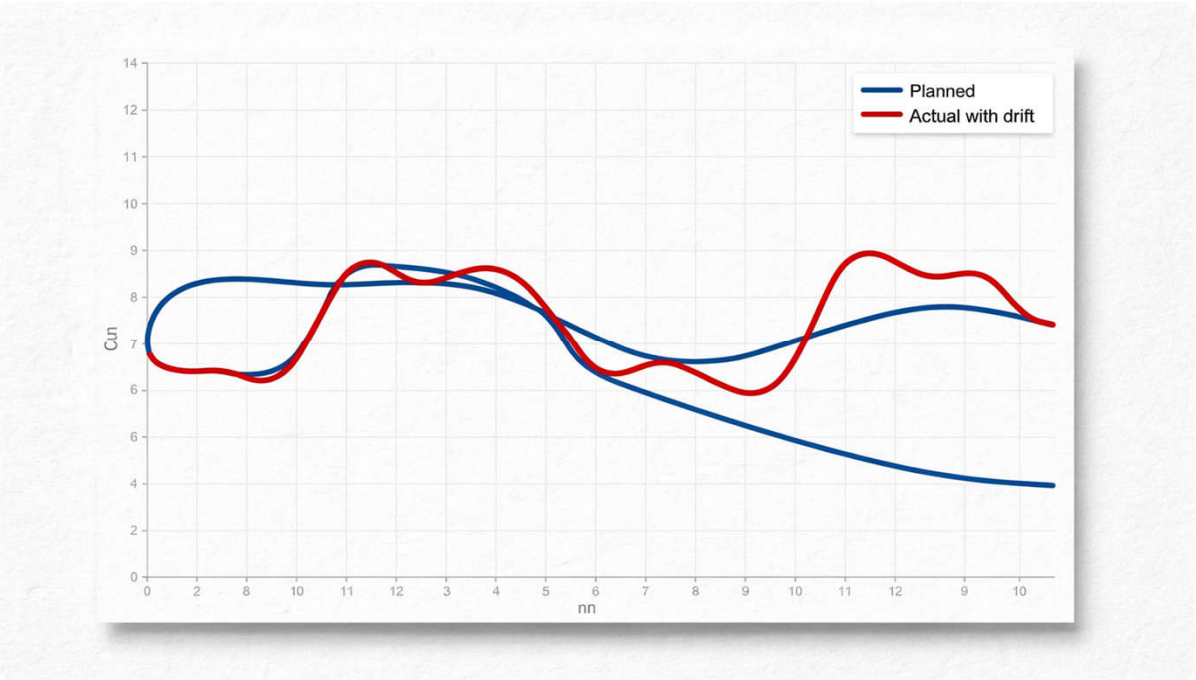
Translation vector — scale is unknown for monocular VO.

Accumulating the Global Trajectory

```
R_global = R_global @ R
t_global = t_global + scale * (R_global @ t)
```

⚠ For monocular VO, the **scale** variable must be obtained from encoders, stereo, or an additional assumption — it cannot be recovered from a single camera alone.

Visualizing the 2D Trajectory



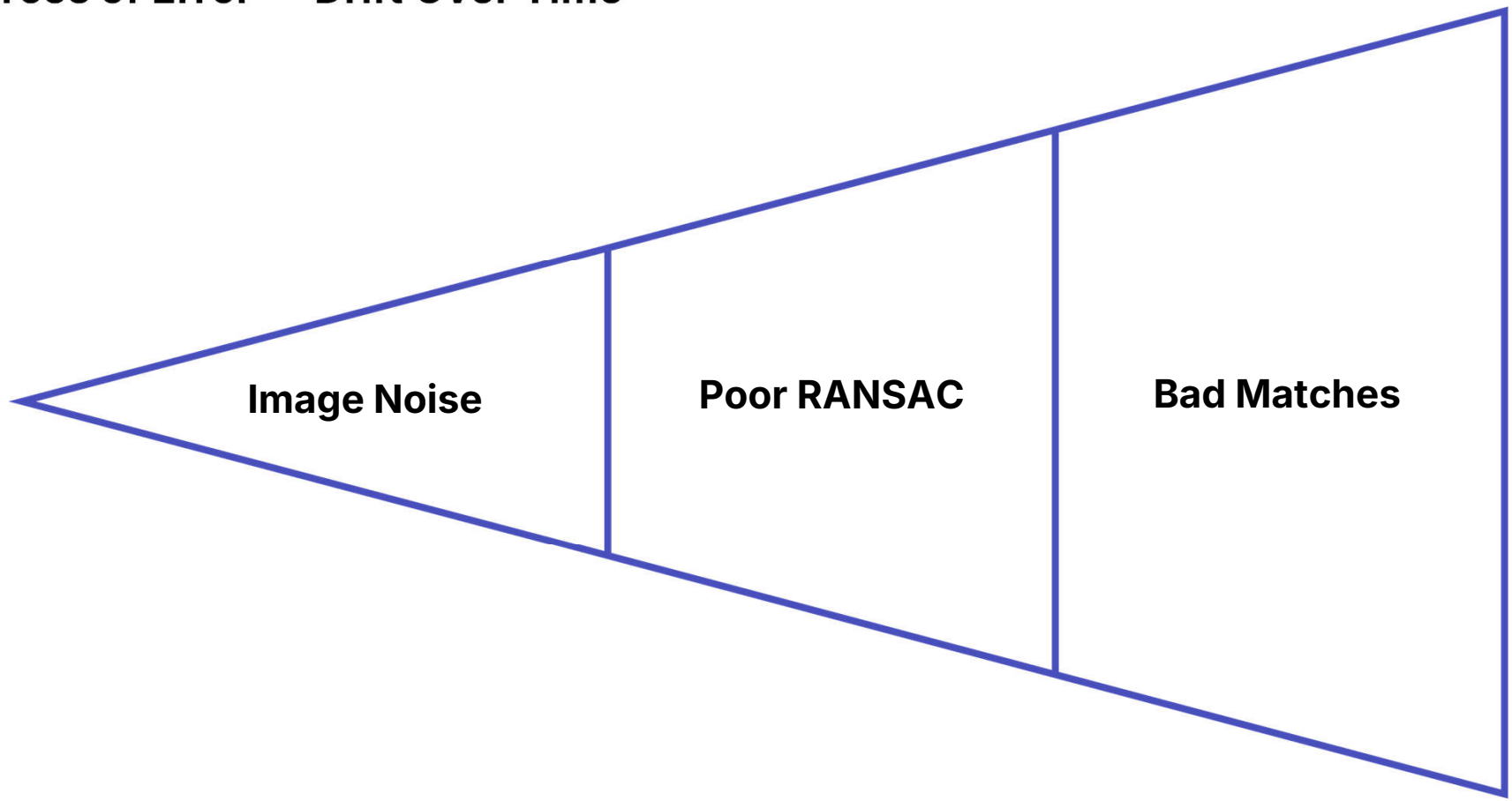
Approach

Project 3D position onto the floor plane — use x-z as the top-view map.

Display

Draw trajectory using `cv2.line` or `matplotlib` so students can observe drift accumulating over time.

Sources of Error — Drift Over Time



Each small error compounds across hundreds of frames — understanding this cascade is key to designing more robust VO systems.

Why Does the Trajectory Drift?

Small Per-Step Errors

Accumulate across hundreds of frames into large deviations.

No Absolute Reference

The system only knows relative motion — no global correction.

Scale Uncertainty

Especially severe in monocular VO without external scale anchoring.

Reducing Drift — Practical Strategies

01

Calibrate camera carefully before deployment.

02

Discard blurry frames before processing.

03

Improve feature matching quality.

04

Fuse encoder and IMU data.

05

Exclude dynamic object regions.

06

Reset using known markers when available.

07

Apply bundle adjustment (advanced).

Turtlebot 4 — Week 12 Application

Camera

Receives consecutive frames for motion estimation.

Encoders

Provides scale and baseline distance traveled.

Python SDK

Sends drive commands and logs pose over time.

LAB 12.1

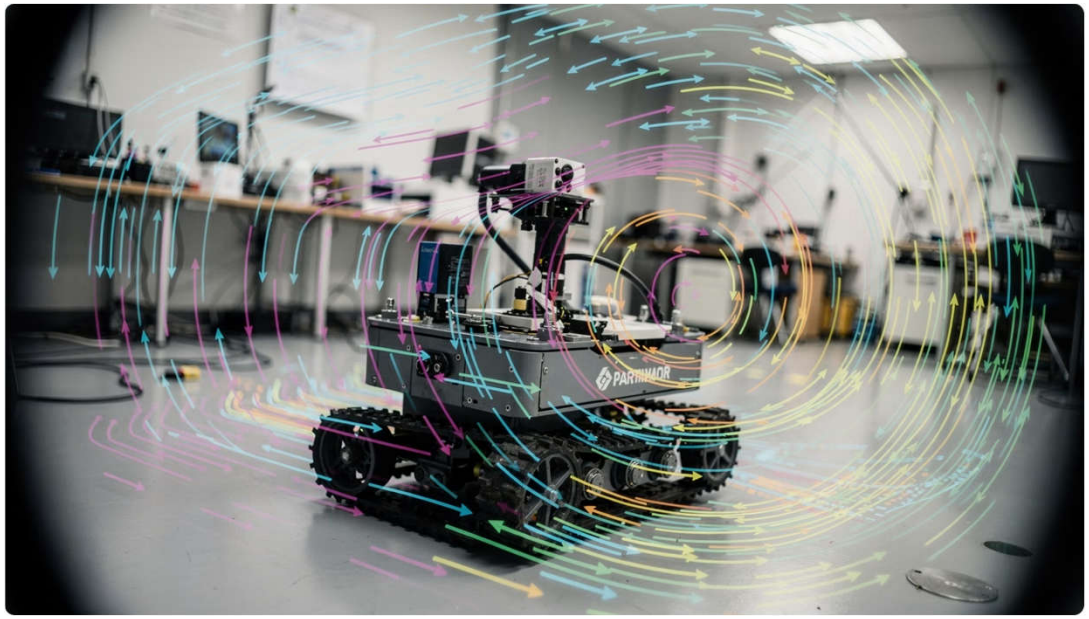
Wheel Odometry Baseline

1. Read ticks from both wheels.
2. Convert ticks to meters.
3. Compute Δs and Δθ.
4. Update 2D pose and plot trajectory.

Goal: Show students that encoders alone can track a path — but drift appears when wheels slip.

LAB 12.2

Optical Flow Tracking



01

Use `goodFeaturesToTrack` to select corners.

02

Use `calcOpticalFlowPyrLK` to track points.

03

Draw motion vectors between two frames.

04

Observe differences between turning and straight motion.

LAB 12.3

Monocular VO with ORB

1

Detect

ORB detect + compute on two frames.

2

Match

BFMatcher to find correspondences.

3

Estimate

`findEssentialMat` + `recoverPose`.

4

Accumulate

Build relative trajectory over time.

LAB 12.4

Fusion: VO + Encoder

Encoder → Scale

Use encoder data to anchor motion magnitude.

Camera → Direction

Use VO to determine relative heading.

Compare

Plot encoder-only vs. fused trajectory side by side.

Analyze

Identify when fusion outperforms encoder alone.

LAB 12.5

Plot Robot on a 2D Map

01

Draw origin, robot heading, and path using matplotlib or OpenCV canvas.

02

Save trajectory as image or video.

03

Analyze error after each run loop.

What Should Students Submit?

Code

Runnable script with comments and data logs.

Images / Video

Evidence of keypoints, matches, and trajectory visualization.

Analysis

Identify error sources, describe observed drift, and propose improvements.

Week 12 → Week 13: The Bridge to Navigation

Without Pose Estimate	With Relative Pose
Robot cannot systematically avoid obstacles or reach a goal.	Robot can begin short-term planning, turning, and updating its local map position.

Visual Odometry is the foundation that makes autonomous navigation possible — Week 13 builds directly on this.

5 Key Takeaways from Week 12

1. **VO is odometry using a camera** to infer relative motion.
2. **Feature-based VO** is the right starting point for AI students.
3. **Monocular VO has scale ambiguity** — always needs a workaround.
4. **Encoder + camera fusion** is highly practical for Turtlebot.
5. **Drift is inherent** — always design a strategy to reduce it.

Week 12 Complete

Visual Odometry for Turtlebot 4

From image features to robot trajectory tracking in Python.